

Clasificador Automático de Frutas con IA

Reporte Técnico — Arquitectura V4 (Agéntico)

Python 3.10+

Arduino UNO R3

LMStudio

VL53L0X Láser

Tool-Calling



Desarrolladores

Jesús Andrés Mondragón Tenorio

Cristofer Piña Rodriguez

Mauricio Sanchez Garcia

Tabla de Contenido

1. Descripción del Sistema	3
1.1 ¿Cómo funciona en términos generales?	3
1.2 Características Principales	3
1.3 Evolución: V3 → V4	3
2. Hardware	4
2.1 Lista de Componentes	4
2.2 Sensor Láser VL53L0X	4
2.3 Conexiones de Pines	4
2.4 Diagrama del Circuito	5
3. Arquitectura del Sistema	6
3.1 Diagrama de Componentes	6
3.2 Flujo de un Ciclo Completo	6
4. Módulos de Software	7
4.1 service.py — Punto de Entrada	7
4.2 llm.py — Agente de IA	7
4.3 tools.py — Herramientas del Agente	8
4.4 arduino.py — Comunicación Serial	8
4.5 camera.py — Captura de Imágenes	8
4.6 mcp_service.py — Servidor MCP (Opcional)	9
5. Comunicación HTTP con requests	10
5.1 ¿Qué hace exactamente?	10
5.2 ¿Por qué requests en lugar del SDK?	10
6. Loop Agéntico y Tool-Calling	11
6.1 Cómo funciona el loop	11
6.2 Estructura del Schema JSON (OpenAI format)	11
6.3 Caso especial: imagen poco clara	11
6.4 Condición de terminación	11
7. Conexión con LMStudio	13
7.1 Configuración	13
7.2 Requisitos del modelo	13
8. Protocolo Serial Arduino	14
8.1 Casos especiales del sensor VL53L0X	14
9. Configuración Rápida	15
9.1 Paso 1 — LMStudio	15
9.2 Paso 2 — Arduino	15
9.3 Paso 3 — Python	15

9.4 Argumentos de Línea de Comandos	15
10. Prototipo Físico	16
10.1 Estructura del Repositorio	16
11. Glosario	17

1. Descripción del Sistema

El **Clasificador Automático de Frutas con IA** es un sistema embebido autónomo que combina visión por computadora, inteligencia artificial local y control de hardware para clasificar frutas en una rampa física. El sistema no requiere conexión a internet ni servicios en la nube — todo el procesamiento ocurre localmente.

1.1 ¿Cómo funciona en términos generales?

Una fruta se coloca sobre una rampa. El sensor láser **VL53L0X** detecta su presencia midiendo la distancia. Python captura una foto con la webcam, la codifica en Base64 y la envía al modelo de lenguaje **Qwen3-VL-4B** ejecutándose en LMStudio. El modelo identifica la fruta y *llama* la herramienta correcta (tool-calling), que envía un comando serial al Arduino para activar el servo correspondiente.

1.2 Características Principales

- **100% local** — Sin dependencias de cloud. LMStudio corre en la misma PC.
- **Agente autónomo** — El LLM decide qué tool llamar; Python solo ejecuta.
- **HTTP estándar** — Usa requests con la API OpenAI compatible.
- **Sensor láser VL53L0X** — Mayor precisión que ultrasónico. I2C, 3–13 cm.
- **Escalable por prompt** — Añadir una fruta nueva solo requiere editar el system prompt.
- **Arquitectura modular** — Cada módulo es independiente y reemplazable.

1.3 Evolución: V3 → V4

Aspecto	V3	V4 (este proyecto)
Sensor de detección	HC-SR04 (ultrasónico, pines 6/7)	VL53L0X (láser I2C, A4/A5)
Comunicación con LLM	LMStudio SDK (WebSocket)	requests HTTP (estándar OpenAI)
API Key	No requerida	Bearer token explícito
Tool-calling	Manejado por el SDK	Loop agéntico manual
Firmware Arduino	Lógica de doble servo (push)	Movimiento simple por servo
Compatibilidad LLM	Solo LMStudio SDK	Cualquier API OpenAI-compatible

2. Hardware

El sistema utiliza componentes de bajo costo y fácil adquisición. El elemento clave es el sensor **VL53L0X**, que usa tecnología de tiempo de vuelo (ToF) láser para detectar la presencia de frutas con mayor precisión y menor ruido que los sensores ultrasónicos.

2.1 Lista de Componentes

Componente	Modelo	Función
Microcontrolador	Arduino UNO R3	Controla servos y lee el sensor láser
Cámara	Webcam USB	Captura imágenes para el agente de IA
Sensor distancia	VL53L0X (Adafruit)	Detección de fruta por láser ToF
Servomotores ×2	MG995 (alto torque)	Accionan las compuertas de clasificación
Estructura	Triplay / MDF	Rampa con compuertas y contenedores

2.2 Sensor Láser VL53L0X

El **VL53L0X** es un sensor de distancia de **Tiempo de Vuelo (ToF)**. Emite pulsos de láser infrarrojo (940 nm) y mide el tiempo que tarda en regresar el reflejo, calculando la distancia con precisión milimétrica. A diferencia del HC-SR04 (ultrasónico), no necesita pines de Trig/Echo — se comunica por **I2C** con solo dos cables (SDA/SCL), es insensible al ángulo de la superficie y funciona bien con objetos blandos como frutas.

Parámetro	Valor
Protocolo de comunicación	I2C (dirección 0x29)
Rango físico del sensor	30 mm – 2000 mm
Rango configurado (firmware)	30 mm – 130 mm (3–13 cm)
Modo de medición	VL53L0X_SENSE_HIGH_ACCURACY
Valor fuera de rango	999.0 (ausencia de objeto o error)
Librería Arduino	Adafruit_VL53L0X
Umbral de detección (Python)	13.0 cm (configurable con --threshold)

2.3 Conexiones de Pines

Pin Arduino	Componente	Función
A4 (SDA)	VL53L0X → SDA	Datos I2C (protocolo bidireccional)
A5 (SCL)	VL53L0X → SCL	Reloj I2C
3.3V	VL53L0X → VCC	Alimentación del sensor (regulador onboard)

Pin Arduino	Componente	Función
9	Servo MG995 #1	Compuerta IZQUIERDA (manzanas)
10	Servo MG995 #2	Compuerta DERECHA (naranjas)
5V	Servos → VCC	Alimentación de los servomotores
GND	VL53L0X / Servos	Tierra común del sistema

2.4 Diagrama del Circuito

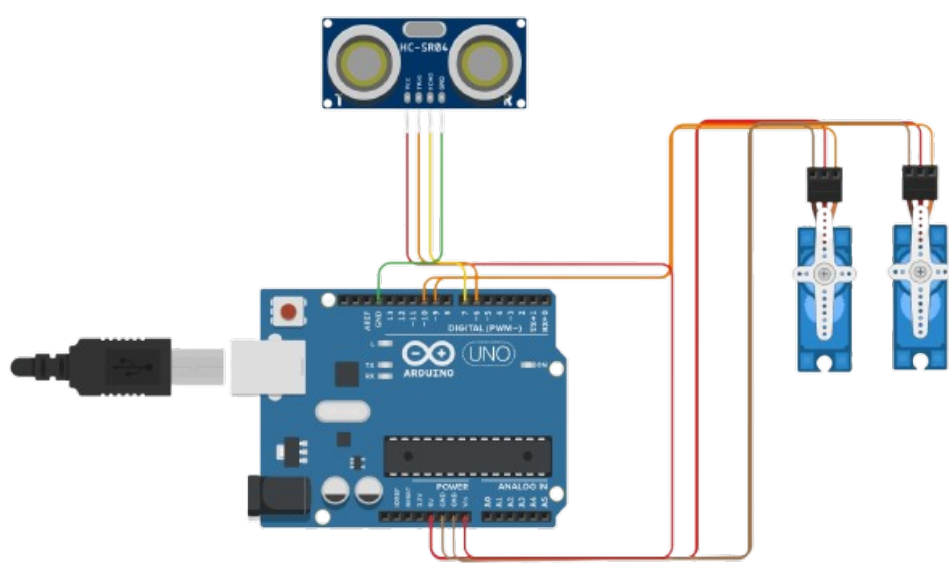


Figura 1 — Diagrama de conexiones del circuito

3. Arquitectura del Sistema

El sistema sigue una arquitectura de **agente autónomo**: Python orquesta el hardware pero *no toma decisiones de clasificación* — esas las toma el LLM mediante tool-calling.

3.1 Diagrama de Componentes

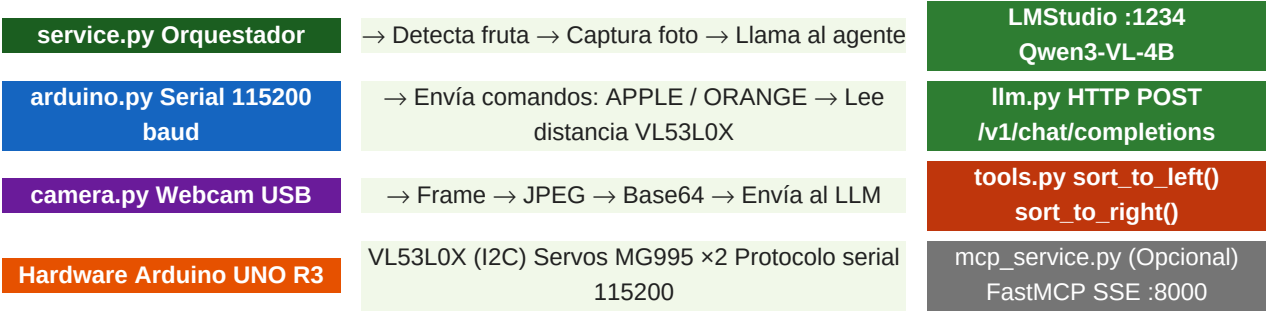


Figura 2 — Relación entre módulos del sistema

3.2 Flujo de un Ciclo Completo

Pas o	Módulo	Acción	Resultado
1	arduino.py	Polling GET_DISTANCE cada 0.3s	Espera distancia < 13 cm
2	arduino.py	Detecta objeto en rango 3–13 cm	Retorna distancia en cm
3	camera.py	Captura frame de webcam	JPEG 384px → Base64
4	llm.py	POST /v1/chat/completions + imagen + tools	LLM procesa la imagen
5	llm.py	LLM devuelve tool_calls JSON	sort_to_left / sort_to_right
6	tools.py	Ejecuta la función Python de la tool	Llama arduino.classify_*)
7	arduino.py	Envía APPLE\n o ORANGE\n por serial	Arduino activa servo
8	Arduino	Servo: 90° → 135°/45° → 90° (2.5s)	Fruta desviada al contenedor
9	service.py	Imprime resultado y reinicia ciclo	SUCCESS:FruitName:SIDE

4. Módulos de Software

El sistema está dividido en 6 módulos Python, cada uno con responsabilidad única. Solo `service.py` se ejecuta directamente — los demás son importados como librerías internas.

4.1 `service.py` — Punto de Entrada

Es el **orquestador principal**. Contiene el loop de clasificación, maneja la señal SIGINT para cierre limpio y muestra el resultado en consola con colores ANSI. **No toma decisiones de clasificación** — delega todo al agente LLM.

`service.py` — `sorting_loop()`

```

1  def sorting_loop(threshold_cm: float = 13.0) -> None:
2  while _running:
3  result = arduino.wait_for_fruit(threshold_cm=threshold_cm, timeout_seconds=30)
4  if not result["success"]:
5  log("■ Tiempo agotado sin detección.", YELLOW)
6  continue
7  img_b64 = camera.get_camera_data()
8  agent_status = llm.act_on_fruit(img_b64, on_message=_on_agent_message)
9  print_result_box(agent_status)
10 time.sleep(1.0)

```

4.2 `llm.py` — Agente de IA

Implementa el **loop agéntico**: envía la imagen al LLM, recibe `tool_calls`, ejecuta las tools y repite hasta que la fruta queda clasificada. Máximo 5 iteraciones. Si el LLM pide `get_camera_image()`, Python *reemplaza* la imagen en el historial (no la añade) para ahorrar tokens de contexto.

`llm.py` — `act_on_fruit()` — Loop agéntico completo

```

1  def act_on_fruit(image_b64: str, on_message=None) -> str:
2  headers = {"Authorization": f"Bearer {LMSTUDIO_API_KEY}"}
3  messages = [
4  {"role": "system", "content": SYSTEM_PROMPT},
5  {"role": "user", "content": [
6  {"type": "text", "text": "Sort this fruit."},
7  {"type": "image_url", "image_url": {"url": f"data:image/jpeg;base64,{image_b64}"}}
8  ]}
9  ]
10 for _ in range(5):
11 payload = {"model": LMSTUDIO_MODEL, "messages": messages,
12 "tools": tools.TOOLS_SCHEMA, "tool_choice": "auto"}
13 response = requests.post(API_URL, headers=headers, json=payload)
14 msg = response.json()["choices"][0]["message"]
15 messages.append(msg)
16 tool_calls = msg.get("tool_calls")
17 if not tool_calls:
18 return msg.get("content", "Process finished.")
19 for tc in tool_calls:
20 func_name = tc["function"]["name"]

```



```

21 args = json.loads(tc["function"].get("arguments", "{}"))
22 result = tools.AVAILABLE_FUNCTIONS[func_name](**args)
23 messages.append({"role": "tool", "content": result,
24 "tool_call_id": tc["id"]})
25 if result.startswith("SUCCESS") or result.startswith("DISCARD"):
26 return result
27 return "ERROR:Max turns reached"

```

4.3 tools.py — Herramientas del Agente

Define las tools disponibles para el LLM en dos partes: la **implementación Python** (lo que se ejecuta) y el **schema JSON** (lo que se le describe al modelo para que sepa cuándo y cómo llamarla). El argumento `fruit_name` permite al LLM reportar explícitamente qué fruta identificó.

Tool	Argumento	Acción en Hardware	Devuelve
<code>sort_to_left()</code>	<code>fruit_name (str)</code>	<code>arduino.classify_as_apple()</code> → APPLE\n	SUCCESS:Nombre:LEFT
<code>sort_to_right()</code>	<code>fruit_name (str)</code>	<code>arduino.classify_as_orange()</code> → ORANGE\n	SUCCESS:Nombre:RIGHT
<code>discard_fruit()</code>	—	Sin acción	DISCARD:Unknown Item
<code>get_camera_image()</code>	—	Captura nueva foto (retry)	NEW_IMAGE_READY

4.4 arduino.py — Comunicación Serial

Gestiona la conexión serial con el Arduino. La conexión es **persistente** (se abre una vez y se reutiliza), **thread-safe** (con `threading.Lock`) y **auto-reconectable** (si el puerto falla, el siguiente comando intenta reconectar). Incluye auto-detección del puerto en Linux y Windows.

```

arduino.py — wait_for_fruit()
1 def wait_for_fruit(threshold_cm=13.0, timeout_seconds=30) -> dict:
2     """Poll VL53L0X each 0.3s until object < threshold_cm or timeout."""
3     start = time.time()
4     while time.time() - start < timeout_seconds:
5         distance = get_distance() # Sends GET_DISTANCE\n → returns float
6         if distance < threshold_cm:
7             return {"detected": True, "distance_cm": distance,
8 "message": "Fruit detected. Ready to classify."}
9         time.sleep(SENSOR_POLL_INTERVAL) # 0.3s
10    return {"detected": False, "distance_cm": 999.0,
11 "message": "Timeout: no fruit detected."}

```

4.5 camera.py — Captura de Imágenes

Mantiene la webcam abierta de forma persistente. Incluye un warmup de 2 frames para evitar capturas oscuras o inestables al inicio. Las imágenes se **redimensionan a ≤384px** antes de enviarlas al LLM para reducir tokens y acelerar la inferencia.

4.6 mcp_service.py — Servidor MCP (Opcional)

■ Este módulo es OPCIONAL. `service.py` NO lo requiere para funcionar. Solo es necesario si quieres conectar un cliente MCP externo (Claude Desktop, LMStudio con MCP, etc.) para controlar la máquina desde otro agente.

Usa **FastMCP** para exponer las mismas funciones de hardware como herramientas MCP con transporte **SSE (Server-Sent Events)** en el puerto 8000. Ejecutar este módulo y `service.py` al mismo tiempo causará conflicto en el puerto serial.

5. Comunicación HTTP con requests

La librería **requests** reemplaza al SDK propietario de LMStudio V3. Cada consulta al LLM es un HTTP POST estándar — el mismo formato que usa la API oficial de OpenAI. Esto hace el sistema compatible con **cualquier servidor LLM local o remoto**.

5.1 ¿Qué hace exactamente?

```
llm.py — Anatomía de la petición HTTP
1  # Cada iteración del loop agéntico envía exactamente esto:
2  response = requests.post(
3      url = "http://127.0.0.1:1234/v1/chat/completions",
4      headers = {
5          "Content-Type": "application/json",
6          "Authorization": "Bearer lm-studio" # ← API Key
7      },
8      json = {
9          "model": "qwen/qwen3-vl-4b",
10         "messages": [ # ← Historial completo
11             {"role": "system", "content": SYSTEM_PROMPT},
12             {"role": "user", "content": [imagen_base64]},
13         # ...resultados de tools anteriores...
14         ],
15         "tools": TOOLS_SCHEMA, # ← Definición JSON de tools
16         "tool_choice": "auto" # ← LLM decide si usar tools
17     }
18 )
19 # La respuesta tiene el mismo formato que la API de OpenAI:
20 msg = response.json()["choices"][0]["message"]
21 tool_calls = msg.get("tool_calls") # ← Qué tools quiere llamar
```

5.2 ¿Por qué requests en lugar del SDK?

Aspecto	SDK LMStudio (V3)	requests HTTP (V4)
Protocolo	WebSocket propietario	HTTP estándar REST
API Key	No requerida	Bearer token explícito
Compatibilidad	Solo LMStudio SDK	Cualquier API OpenAI-compatible
Transparencia	Oculto el protocolo	Se ve exactamente qué se envía
Debuggear	Difícil (WebSocket binario)	Fácil (JSON legible en logs)
Dependencias	lmstudio Python SDK	Solo requests (stdlib casi)

6. Loop Agéntico y Tool-Calling

El **tool-calling** es el mecanismo que permite al LLM invocar funciones Python durante su proceso de razonamiento. En lugar de solo generar texto, el modelo puede "llamar" una función con argumentos estructurados. Python ejecuta la función y devuelve el resultado al modelo para que continúe.

6.1 Cómo funciona el loop

Iteración	Python envía	LLM responde	Python hace
1	Imagen + TOOLS_SCHEMA	tool_calls: sort_to_left("Red Gala Apple")	Ejecuta sort_to_left()
—	Tool result: SUCCESS:Red Gala Apple:LEFT	(fin — no más tool_calls)	Termina el loop ■

6.2 Estructura del Schema JSON (OpenAI format)

```
tools.py — TOOLS_SCHEMA (formato OpenAI)
1 # tools.py — Así se describe sort_to_left() al LLM:
2 {
3   "type": "function",
4   "function": {
5     "name": "sort_to_left",
6     "description": "Sort the detected fruit to the LEFT bin (e.g., for apples).",
7     "parameters": {
8       "type": "object",
9       "properties": {
10        "fruit_name": {
11          "type": "string",
12          "description": "The specific name of the fruit (e.g., 'Red Gala Apple')."
13        }
14      },
15      "required": ["fruit_name"]
16    }
17  }
18 }
```

6.3 Caso especial: imagen poco clara

Si la primera foto no es suficientemente clara, el LLM puede llamar `get_camera_image()` para pedir otra. En ese caso, Python captura un nuevo frame y **reemplaza** la imagen original en el historial de mensajes (en lugar de añadir un mensaje nuevo). Esto evita duplicar tokens de imagen en el contexto.

6.4 Condición de terminación

El loop termina cuando el resultado de una tool empieza con "SUCCESS:" o "DISCARD:". Si ninguna tool devuelve uno de estos prefijos en 5 iteraciones, se devuelve "ERROR:Max turns reached".

7. Conexión con LMStudio

LMStudio actúa como un servidor HTTP local que expone una API **100% compatible con OpenAI** en el puerto 1234. No hay ninguna diferencia en el código Python entre conectarse a LMStudio local o a la API de OpenAI real.

7.1 Configuración

Parámetro	Valor	Dónde se define
URL base	http://127.0.0.1:1234/v1	llm.py → API_URL
Modelo	qwen/qwen3-vl-4b	llm.py → LMSTUDIO_MODEL
API Key	lm-studio	llm.py → LMSTUDIO_API_KEY
Endpoint usado	/v1/chat/completions	llm.py → act_on_fruit()
Puerto del server	1234	LMStudio Desktop → Developer

7.2 Requisitos del modelo

El modelo debe soportar **visión (VLM)** y **tool-calling**. Qwen3-VL-4B es el modelo recomendado por su balance entre precisión y velocidad de inferencia local.

- El servidor LMStudio debe estar activo antes de ejecutar service.py.
- En LMStudio Desktop: **Developer** → **Local Server** → **Start Server** (puerto 1234).
- El sistema verifica la conexión con `llm.test_connection()` antes de iniciar el loop.
- Si LMStudio no responde, service.py muestra error y sale con código 1.

8. Protocolo Serial Arduino

La comunicación entre Python y el Arduino es texto plano a **115200 baud**. Cada comando es una línea terminada en `\n`. La respuesta también es una línea terminada en `\n`.

Comando enviado	Respuesta esperada	Acción en Arduino
PING	PONG	Verificación de conexión
GET_DISTANCE	"12.34"	Lee VL53L0X, devuelve distancia en cm (float)
APPLE	OK	Servo izq.: 90°→135°, espera 2.5s, →90°
ORANGE	OK	Servo der.: 90°→45°, espera 2.5s, →90°
(cualquier otro)	ERROR:UNKNOWN_COMM AND	Comando no reconocido

8.1 Casos especiales del sensor VL53L0X

Condición	Valor devuelto	Significado
RangeStatus == 4	999.0	Objeto fuera del campo visual del sensor
Lectura < 30 mm	999.0	Muy cerca — fuera del rango mínimo configurado
Lectura > 130 mm	999.0	Muy lejos — fuera del rango máximo configurado
Lectura 30–130 mm	X.X cm	Objeto detectado en zona válida

■ Al encender el Arduino, espera el mensaje **READY** en el monitor serial antes de ejecutar `service.py`. Si el VL53L0X no inicializa, el firmware envía **ERROR:SENSOR_INIT** y entra en loop infinito — revisa las conexiones I2C.

9. Configuración Rápida

9.1 Paso 1 — LMStudio

- Descarga e instala LMStudio desde [lmstudio.ai](#).
- Carga el modelo con soporte de visión y tool-calling: qwen/qwen3-vl-4b.
- En LMStudio Desktop: **Developer** → **Local Server** → **Start Server** (puerto 1234).

9.2 Paso 2 — Arduino

- Instala la librería Adafruit_VL53L0X desde el Library Manager del IDE de Arduino.
- Conecta los componentes según la tabla de pines (Sección 2.3).
- Carga `fruit_sorter_nuevo.ino` en el Arduino UNO.
- Verifica en el Monitor Serial (115200 baud) que aparezca `READY`.

9.3 Paso 3 — Python

Terminal — Instalación y ejecución

```
1 # Instalar dependencias
2 python3 -m pip install -r requirements.txt
3
4 # Ejecutar el sistema
5 python3 service.py --port /dev/ttyUSB0 --camera 0 --threshold 13.0
```

9.4 Argumentos de Línea de Comandos

Argumento	Default	Descripción
--port PORT	Auto-detect	Puerto serial (ej. /dev/ttyUSB0 o COM5)
--camera INDEX	0	Índice de cámara USB
--threshold CM	13.0	Distancia máxima de detección en cm

10. Prototipo Físico

La maqueta está construida en triplay/MDF con una rampa central, dos compuertas laterales accionadas por los servos MG995, y dos contenedores: izquierdo para manzanas y derecho para naranjas. La cámara se monta sobre la rampa apuntando hacia abajo, y el sensor VL53L0X se coloca en el punto de detección.



Figura 3 — Prototipo físico del Clasificador de Frutas con IA

10.1 Estructura del Repositorio

Estructura de directorios

```
1 Clasificador-de-frutas-/
2
3   ■■■ service.py # ■ PUNTO DE ENTRADA – loop de detección
4   ■■■ llm.py # Agente HTTP – loop agéntico via requests
5   ■■■ tools.py # Tools del LLM – schemas JSON + funciones Python
6   ■■■ arduino.py # Comunicación serial persistente (VL53L0X + servos)
7   ■■■ camera.py # Captura de imágenes webcam (conexión persistente)
8   ■■■ mcp_service.py # (Opcional) Servidor MCP via FastMCP SSE :8000
9   ■
10  ■■■ fruit_sorter_nuevo.ino # Firmware Arduino: VL53L0X I2C + 2x servo MG995
11  ■■■ generate_pdf.py # Este script – genera el reporte técnico
12  ■■■ requirements.txt # Dependencias Python
13  ■
14  ■■■ assets/
15  ■■■ circuit_diagram.png
16  ■■■ maqueta.jpeg
17  ■■■ demo.gif
```

11. Glosario

Definición de los términos técnicos utilizados en el proyecto.

VL53L0X

Sensor de distancia de Tiempo de Vuelo (ToF) fabricado por STMicroelectronics. Emite pulsos láser infrarrojos de 940 nm y mide la distancia calculando el tiempo que tarda la luz en regresar. Interfaz I2C, dirección 0x29.

ToF (Time of Flight)

Técnica de medición de distancia que calcula el tiempo que tarda una señal (luz, sonido) en ir al objeto y regresar. Ofrece alta precisión y es insensible al color o material de la superficie.

I2C (Inter-Integrated Circuit)

Protocolo de comunicación serial de dos cables: SDA (datos) y SCL (reloj). Permite conectar múltiples dispositivos en el mismo bus con distintas direcciones. En Arduino UNO: SDA = A4, SCL = A5.

LMStudio

Aplicación de escritorio para ejecutar modelos de lenguaje grande (LLM) localmente en la PC. Expone una API HTTP compatible con el estándar OpenAI en el puerto 1234.

LLM (Large Language Model)

Modelo de inteligencia artificial entrenado en grandes volúmenes de texto. Capaz de entender y generar lenguaje natural, razonar y, en versiones modernas, interpretar imágenes y llamar funciones (tool-calling).

VLM (Vision Language Model)

LLM con capacidad de procesar imágenes además de texto. En este proyecto se usa Qwen3-VL-4B, que recibe la imagen de la fruta codificada en Base64.

Tool-Calling

Mecanismo que permite a un LLM "llamar" funciones externas durante su respuesta. El modelo devuelve JSON con el nombre de la función y sus argumentos; Python lo ejecuta y devuelve el resultado al modelo para que continúe.

MCP (Model Context Protocol)

Estándar abierto de Anthropic para exponer herramientas, recursos y prompts a modelos de IA. Permite que agentes externos controlen el sistema sin modificar el código Python principal.

SSE (Server-Sent Events)

Protocolo HTTP unidireccional donde el servidor envía eventos al cliente en tiempo real. Usado por mcp_service.py como transporte para el servidor MCP.

Base64

Esquema de codificación que convierte datos binarios (como imágenes JPEG) a texto ASCII. Permite enviar imágenes dentro de un JSON sin caracteres especiales.

PWM (Pulse Width Modulation)

Señal de control para servomotores. El ancho del pulso (entre 1ms y 2ms) codifica el ángulo deseado. La librería Servo de Arduino genera esta señal automáticamente en los pines 9 y 10.

Baud Rate

Velocidad de comunicación serial en bits por segundo. El Arduino y Python deben usar el mismo valor. En este proyecto: 115200 bps.

API Key (Bearer Token)

Credencial de autenticación enviada en el encabezado HTTP Authorization. LMStudio acepta cualquier valor cuando corre localmente; se usa "lm-studio" para cumplir con el estándar.